

LM-based Chess Agents with Human-like Play and Controllable Playing Strength

Kamil Michalak¹, Paweł Rychlikowski¹,

¹University of Wrocław

Abstract

The development of chess engines is a popular topic in artificial intelligence. Modern state-of-the-art engines play at a significantly higher level than even the best human grandmasters. While these engines can often be configured to play at weaker levels (e.g., by limiting search depth), such configurations typically do not result in human-like play. In contrast, the development of agents that emulate human playing styles remains relatively uncommon.

In this paper, we demonstrate that autoregressive language models — without any architectural modifications — can be trained from scratch to serve as chess agents that mimic human-like play. Our approach involves predicting the next move in a game based on the preceding sequence of moves. We introduce a novel tokenization method in which each move is represented by a single token. Through our experiments, we find that: (1) this method enables the training of models that predict human moves with high accuracy; (2) incorporating a one-token representation of a player’s rating allows a single model to emulate players across various skill levels; (3) adding compact, single token-based annotations of the board state improves predictive performance; (4) setting higher ratings for the model yields predictions more similar to those of top chess engines; and (5) combining a language model with a minimax-based agent further enhances move prediction accuracy.

Our findings suggest that training language models to play games in a human-like manner is a promising direction for future research.

1 Introduction

Development of chess-playing AI has a long history. First chess program, *Turochamp*, was developed in 1948 by Alan Turing and David Champernowne. It played (and loose) only one game, but the history started. In 1996 IBM’s *Deep Blue* defeated then-world champion Garry Kasparov in a six-game match by 3.5-2.5, becoming the first chess program to defeat a reigning world champion. The growth of the chess engines’ playing strength continued and current engines, such as Stockfish or AlphaZero, are far more powerful than any human player. While the strength of these engines is impressive, their usefulness for most of the players is limited.

It doesn’t make much sense to use them as sparring partners, since they can easily defeat any player. Usually there are ways to weaken the engine, for example Stockfish search depth can be reduced, or the agent performs occasionally a blunder (i.e. a very weak move). This kind of methods indeed make the engine weaker but doesn’t make it play more like human. On the contrary, reducing the engine’s strength can also reduce its similarity to a human player (McIlroy-Young et al. 2020). If a human player wants to, for example, practice playing against other players on a specific level (e.g. slightly higher than his own), using weakened engines won’t be effective because they play different moves than humans.

There is a need for chess AI that plays like a human and some attempts were made to create it, with a most recent (and most efficient) being Maia (McIlroy-Young et al. 2020) and Maia-2 (Tang et al. 2024). These engines use deep convolutional neural networks trained on datasets of online games, similar architecture (although trained through self-play) was also used in AlphaZero (Silver et al. 2018).

We present a different approach that is based on language models. In recent years transformer-based language models (Vaswani et al. 2017) significantly outperformed previous architectures in natural language processing and became state-of-the-art technology. They were also successfully used outside of NLP, for example in image generation (Ramesh et al. 2021) music composition (Wu et al. 2023) or different time series forecasting (Wu et al. 2021). Some research on training generative pre-trained transformers (GPT) to play chess has been done in recent years (November, Ciolino, and Kalin 2020; Toshniwal et al. 2022), but the main goal of this research was to explore the possibility of learning the rules of chess by a transformer-based autoregressive language model, rather than building a meaningful playing agent. Our goal was to improve this methods and create a full-fledged GPT-based chess agent that plays in a human-like fashion. Our contribution is as follows:

1. We introduce our original method for representing a chess games as a sequence of tokens. We show that a language models based on GPT architecture can be trained on a dataset consisting of such sequences (from on-line games) to predict chess moves of human players with high accuracy and therefore can serve as a chess agents with a human-like style of play. This does not require any changes to the architecture, only data preparation in

a specific format is required. Note that we only use GPT architecture but not any pretrained weights. Our models are trained from scratch.

2. We show that including an information about players' ratings into a training data allows to train a single model with adjustable playing strength, where we can set the model to play like a human with a specific rating.
3. We introduce annotations - special tokens placed after each move with some information about the game state. We show that including these annotations into a data improve the models' prediction accuracy.
4. We show that if we set the model's rating to a higher value, its predictions become more similar to the moves played by Stockfish.
5. We show that combining a language model with minimax-based agents improve the prediction accuracy.

2 Related Work

2.1 Chess AI

Historically, most of the chess engines were based on a search algorithm (such as minimax) combined with a hand-crafted function for static position evaluation. This was the case with Deep Blue (Hsu 1999), as it is today with a classic version of Stockfish, one of the strongest chess engines.

For a long time, the use of neural networks in the implementation of chess programs seemed to be a less attractive alternative due to runtime performance. This was because agents based on search derived their strength from analyzing millions of game states that was unattainable for programs using neural networks. The first approach that produced a strong neural network-based agent was the DeepChess program (David, Netanyahu, and Wolf 2017), implemented in 2016. The program played at a master level and was able to beat older "traditional" programs; nevertheless, it was still far from the strongest players.

In 2018, Yu Nasu, in the context of traditional Japanese game, proposed NNUE (efficiently updatable neural networks) (Nasu 2018), which could be used in chess engines without losing the ability to efficiently search through a vast number of game states. They are now part of agents such as Stockfish.

Another possibility is the use of deep convolutional neural networks, which enabled the development of programs like AlphaZero (Silver et al. 2018) and Leela (Pascutto, Gian-Carlo and Linscott, Gary 2019), achieving excellent results despite searching through several orders of magnitude fewer states. A novel aspect of both these programs was the training method, based entirely on self-play games, where the training agent played against itself.

A different approach is presented by the authors of the Maia (McIlroy-Young et al. 2020) and Maia-2 (Tang et al. 2024) projects. Their goal is not to maximize the playing strength of the agents, but to faithfully reproduce human playing style (measured by the accuracy of the human moves prediction).

An excellent and accessible summary of neural networks in chess can be found in the book (Klein 2022).

2.2 Autoregressive Language Models

Autoregressive language models compute the probability distribution of the tokens t_n conditioned (theoretically) on all the previous tokens, i.e. $P(t_n | t_1 \dots t_{n-1})$. Since approx. 2010 this distribution is parametrized by the neural network. Several architectures were used in this task, including fixed horizon simple neural network (Bengio, Ducharme, and Vincent 2000), causal convolutional networks (Dauphin et al. 2017), and recursive neural networks (Zaremba, Sutskever, and Vinyals 2014). Nowadays, virtually all popular language models use transformer networks (Vaswani et al. 2017), and its most common, decoder-only variant introduced in introduced in (Radford and Narasimhan 2018). In this article, we also use this variant, based on the nanoGPT¹ implementation.

2.3 Language Models in Board Games

Language models were used to predict valid moves in Othello (Li et al. 2022), and (Hazineh, Zhang, and Chiu 2023), but the authors concentrated only on the correctness of moves (which is much harder in this game than in chess, due to the massive changes of the game state after each move).

In (Noever, Ciolino, and Kalin 2020) GPT2 transformer was fine-tuned to generate chess games in PGN format. In (Toshniwal et al. 2022) GPT2 was trained from scratch to model chess games. Authors used their own tokenization method that was based on UCI notation. In their research each token represented a single square, a type of moved piece or a piece that a pawn is promoted to. Depending on the variant of notation (multiple variants were tested), each single move required 2 to 4 tokens.

3 Our Approach

Each chess-playing program can be regarded as a function $\text{BestMove} : \text{GameStates} \rightarrow \text{Moves}$, which returns the best move for a given position on the board. If we assume that a game state is defined by the sequence of moves that led to that state, we can then use any autoregressive language model trained on real move sequences as a game playing agent. This is the starting point of our approach. We demonstrate the possibilities it offers and how they can be enhanced by adding additional information to the move sequences.

3.1 Games Tokenization

We trained our models using our novel tokenization method. We represent chess games using Universal Chess Interface (UCI)² notation with additional information what piece is being moved. Each chess move is represented by exactly one token with a piece symbol, starting square and ending square. For example, token `Bf1c4` represents a bishop move from f1 to c4. Pawn promotions are represented by tokens with an additional symbol of the piece to which the pawn is promoted (a lowercase letter). For example, token `Pe7e8q` represents a pawn move from e7 to e8 with promotion to a queen. The vocabulary contains only tokens of

¹<https://github.com/karpathy/nanoGPT>

²For more details see https://en.wikipedia.org/wiki/Universal_Chess_Interface

moves that are legal in some position, so there are no tokens that represent e.g. a queen moving like a knight. Summary of these move tokens is presented in Table 1. We think this tokenization is better than the one used in (Toshniwal et al. 2022), where each move was represented by 2-4 tokens, for several reasons.

First, it makes it easier to implement a chess agent that uses a language model. With our approach we need only one prompt to get probabilities for each possible next token. Then our agent can simply play a legal move with highest probability according to the model. We can also implement an agent that takes probabilities of all legal moves, adjusts them, so they add up to one, and plays a random move according to these probabilities. To get the same information from a model with tokenization from (Toshniwal et al. 2022), we need to feed it with multiple prompts and multiply probabilities for each set of tokens that represent each single move. Our idea is much more efficient and less prone to implementation errors.

Second, it should make it easier for the model to track the board state. In (Toshniwal et al. 2022), a model has to analyze several tokens and link them to each other to understand each move. With our approach, a model gets all this information in one token.

Third, we think that our tokenization reduces the chance of generating something that is not a valid chess game. In (Toshniwal et al. 2022), it is possible to generate moves that can never be legal (e.g. $[N, a1, a2]$ – knight from a1 to a2) or something that can’t even be considered a move, legal or not (e.g. $[q, K, e4]$). This type of errors are impossible with our tokenization.

3.2 Elo Tokens

We added information about players’ Elo rating to our notation. A tokenized game starts with two Elo tokens for white and black player. Each Elo token contains player rating rounded down to the nearest hundred, so token 1500 means that a player’s Elo was between 1500 and 1599 during this game. Adding elo tokens let us train a single model that is able to play like humans at different skill levels. For most of our experiments, we randomly masked one of the Elo tokens during training (see Table 3). Suppose that we want to have an agent that plays like 1500-rated human player. If the agent has to play as white in a given position, it only need to feed the model with the game history prefixed by Elo tokens, where black’s token is masked e.g. 1500 [UNK_ELO] Pe2e4 Pc7c5.

3.3 Game State Annotations

We also trained separate models on the same games, but using an extended notation with annotations: special tokens placed after each move (one annotation token per move token), which contain information about the game state and are intended to help the model maintain an accurate representation of the board state. We tested two annotation variants encoding information about the current player’s material.

In the material sum variant, each annotation represents the total value of all the player’s pieces according to a standard

Type	Examples	Count
King moves	Ka1a2, Kf8g7	420
Queen moves	Qd1h6, Qd8e7	1456
Rook moves	Ra1b1, Re8e1	896
Bishop moves	Bf1g5, Bg7b2	560
Knight moves	Ng1f3, Nb7c6	336
Pawn moves	Pe2e4, Pb4c3	236
Castles	Ke1g1, Ke8c8	4
Pawn promotions	Pe7e8q, Pa2b1n	176
Total:		4084

Table 1: Model vocabulary: chess moves

Type	Examples	Count
Player elo	[UNK_ELO], 1000, 2200	41
Material sum	12, 39	104
Piece count	Q1R2B2N2P8	1728
Special symbols	[EOS], [BOS], [PAD]	3

Table 2: Model vocabulary: other tokens

value assignment (queen = 9, rook = 5, bishop = 3, knight = 3, pawn = 1). In the piece count variant, each annotation specifies the exact number of pieces of each type that the player has, e.g., Q1R1B2N1P4, indicating 1 queen, 1 rook, 2 bishops, 1 knight, and 4 pawns.

Since it is possible—although unlikely—for a player to have more pieces of a given type than in the starting position (e.g., due to promotions), we cap the counter for each piece type at the initial count plus one. For example, Q2 denotes that a player has two or more queens. After White’s move, we include annotations only for Black’s material, as White’s move can change only Black’s material through captures; pawn promotions are rare, and all relevant information about them is already encoded in the move tokens.

Our models were trained exclusively to predict chess moves; Elo tokens and annotation tokens were excluded from the loss computation.

4 Technical Details

4.1 Data

For training and testing we used chess games played by humans on Lichess.org, a popular free online chess platform. All games can be downloaded in PGN format from Lichess Database³. We also used Lichess Elite Database⁴ which contains games of higher-rated players. Lichess games are divided into categories depending on how much time each player has to play the game, including Bullet (less than 3 minutes), Blitz (3-10 minutes), Rapid (10-30 minutes) and Classical (30+ minutes). For our research we excluded the fastest Bullet games, since players are often more focused on not losing on time than playing good moves. We used games of length between 2 and 150 moves (4 - 300 ply).

We created training dataset with 8 million games, and validation and test sets with 100,000 games each. We aimed for

³<https://database.lichess.org/>

⁴<https://database.nikonoel.fr/>

Variant	Example
Base	[BOS] Pe2e4 Pd7d5 Pe4d5 Qd8d5 ... Ph3h2 Bb6c7
Elo	[UNK_ELO] 2700 Pe2e4 Pd7d5 Pe4d5 Qd8d5 ... Ph3h2 Bb6c7
Elo (no mask)	2600 2700 Pe2e4 Pd7d5 Pe4d5 Qd8d5 ... Ph3h2 Bb6c7
Material sum	[UNK_ELO] 2600 Pe2e4 39 Pd7d5 39 Pe4d5 38 Qd8d5 38 ... Ph3h2 10 Bb6c7 7
Piece count	[UNK_ELO] 2600 Pe2e4 Q1R2B2N2P8 Pd7d5 Q1R2B2N2P8 Pe4d5 Q1R2B2N2P7 Qd8d5 Q1R2B2N2P7 ... Ph3h2 Q0R1B1N0P2 Bb6c7 Q0R1B0N0P2

Table 3: Notation variants. Each row presents the same game between players rated 2612 and 2729

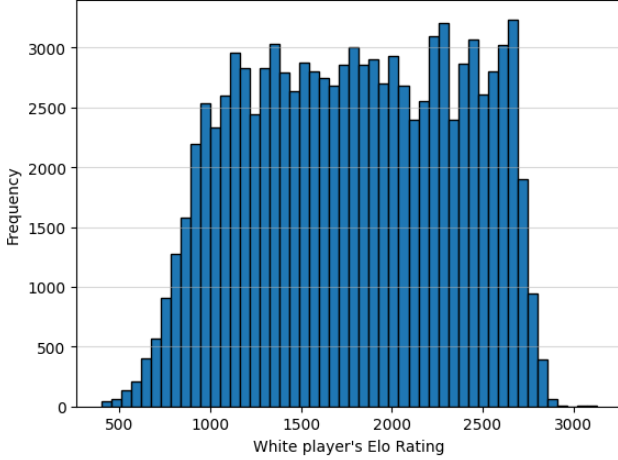


Figure 1: Distribution of white player's Elo in the test set

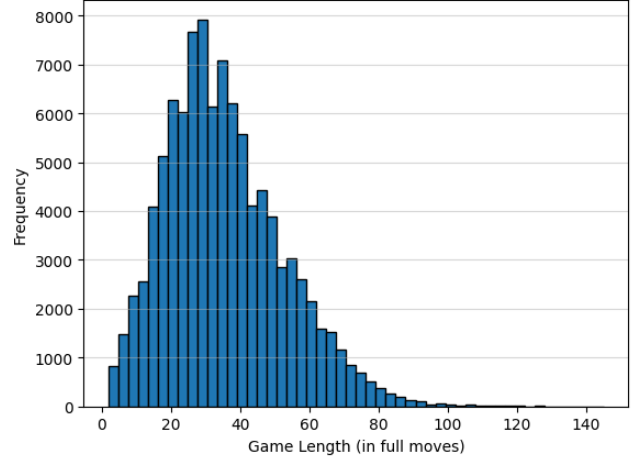


Figure 2: Distribution of game length in the test set

the datasets to have a close-to-uniform distribution of players' ratings between 1000 and 2800 and some games beyond this range. The distribution of white players' Elo in the test set can be seen in Figure 1. Training and validation sets have similar distribution. Figure 2 shows the game length distribution.

4.2 Model Details

For our research we used NanoGPT (Karpathy 2022) – an open-source implementation of GPT in PyTorch. The size of the model can be configured using `GPTConfig` class. Our trained models have 8 layers, 8 attention heads and embedding size of 512 dimensions, bias was set to False and dropout to 0. The remaining parameters varied depending on the notation variant used by a particular model. Block size was set to 604 for models that use annotations and to 302 otherwise. Vocab size varied from 4087 for base notation to 5857 for "piece count" notation. This gives our models between 27.3 and 28.2 million parameters.

4.3 Training Details

We trained a separate model for each notation variant presented in 3. Each model was trained for 10 epochs with a batch size of 64. For optimization we used Adam (Kingma and Ba 2014) with learning rate of 1×10^{-4} . Training and testing was done using Pytorch Lightning framework (Falcon and The PyTorch Lightning team 2019). We used a single NVIDIA RTX A4000 GPU. Training of each model took

from 15 to 24 hours.

5 Experiments

5.1 Prediction Quality

Accuracy and Perplexity We measured prediction quality of our models using top-1 accuracy and perplexity. Accuracy tells us how often the model predicted (assigned the highest probability) to a token that represents a correct human move. Perplexity measures the model's confidence in its predictions. A lower perplexity indicates the model is more confident and accurate in human move prediction, as it corresponds to a higher likelihood of the correct human move. We measured these two metrics for the entire test dataset. We tested each model using the same data format that it was trained on. If a model was trained with elo tokens and masking, then we fed it with each game twice. We masked black's elo to get predictions of white's moves and vice versa. We measured only the prediction of moves, predictions of rating and annotation tokens were ignored.

The results are shown in Table 4. All of our models achieved high accuracy above 50%. Base model that doesn't use Elo tokens achieved the worst result, which was expected. Results of both model with additional rating information show slight improvement over the base model in both metrics. These models achieved very similar results, which indicates that showing to a model ratings of both players instead of one during training and testing doesn't give it any

Notation	Accuracy (%)	Perplexity
Base	51.00	4.59
Elo	51.50	4.50
Elo (no mask)	51.45	4.51
Material sum	52.10	4.40
Piece count	52.19	4.38
Maia-2	53.25	4.07

Table 4: Models move prediction accuracy and perplexity for the entire test dataset

boost in predictive performance. Models trained with annotations achieved the best results, with model with "piece count" notation being slightly better. There is a greater difference in performance between models trained with and without annotations than between models trained with and without ratings. This means that providing additional information about the game state, gives the model a greater performance boost than providing information about the players' rating.

Our best model with "piece count" notation achieved accuracy of 52.19%. This is about 1 p.p. worse than Maia-2, which achieved 53.25%, and is currently the best model for predicting human chess moves. These results are not completely comparable, since different datasets were used for testing, but they indicate that our model's performance is close to the SOTA models. Maia-2 was trained on 169 million games, which is over 20 times larger than our dataset, which has 8 million games. Also, Maia-2 dataset contains only rapid games and they only used moves where both players had at least 30 seconds left on the clock. Our dataset contains games of various time controls (but mainly blitz games) and we didn't discard the moves after 30s threshold. Our dataset contains many moves done under time pressure which are often worse and more random, so it is reasonable to assume that our dataset has a higher variance than Maia-2 dataset and therefore is more difficult to learn.

Accuracy Across Ratings Our goal requires that the model should be able to predict the moves of human players across a broad range of skill levels. To test this, we measured each model's accuracy within rating intervals of 100 points, ranging from 600 to 3000. We also measured accuracy for each combination of players' rating intervals that occurred in the test dataset.

Figure 3 shows prediction accuracy for each rating interval. We found that:

1. All of the models achieved high accuracy across broad range of ratings, above 50% between ratings of 1100-1300 and 2800. Accuracy drops significantly for ratings above 2800, which was expected because there aren't many games of players with such a high rating in the training dataset.
2. Base model has clearly the worst predictive performance. It has only slightly lower accuracy for rating close to the median of the dataset (which is 1797), but the gap is larger for higher and lower ratings, which shows that rating tokens are the most helpful in predicting moves of

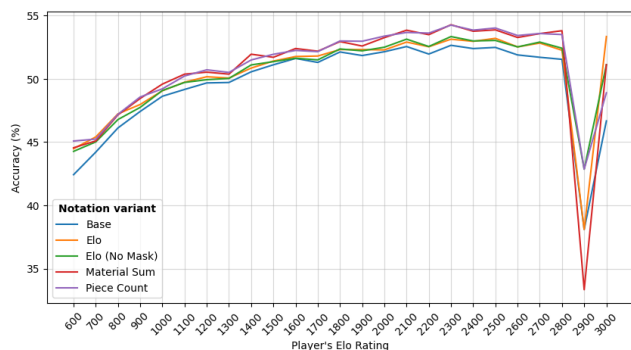


Figure 3: Move prediction accuracy across diverse skill levels

very strong and very weak players.

3. Models with annotations are the most efficient across the entire range of ratings, but their advantage is the largest for games of higher rated players. This might be due to the fact that such games are longer on average and they are more difficult to track for the model, so the annotations become more helpful.

Figure 4 shows accuracy of the models for each combination of rating intervals present in the test dataset. We observe that the models are most accurate in games where both players have similar ratings. The accuracy is lower when there is a significant rating difference between the players. This is expected, because games with large rating difference between are quite rare and they constitute only a small part of the training dataset. We also observe that, in these games, the difference between base model and other models is usually the largest, which indicates that information about rating is particularly useful in such scenarios.

Accuracy Change Over Game Progress It is important for our models to keep high prediction accuracy over the entire course of the game. To test it, we measured the average accuracy over each interval of 5 moves (10 ply). Results are shown in Figure 5. We found that each model keeps high accuracy over 50% for at least 85 moves, which is longer than 99% of games in the dataset. This means that our models can play the entire game if it is not exceptionally long. Performance drops for later moves, but such long games are very rare, so the models didn't have enough training data to learn them. Another notable finding is that the models are more accurate for moves in the range of 45-70 than for earlier moves. We attribute this to the fact that these moves almost always occur during the endgame (73% of games are shorter than 45 moves, and 96% are shorter than 70), when there are fewer pieces on the board and fewer legal options, making the moves easier to predict.

5.2 Quality of Predicted Moves Across Ratings

The model should on average predict better chess moves when it is given a higher rating since higher ranked chess players generally plays better moves. To test this we selected 5000 games from the test dataset and checked what move

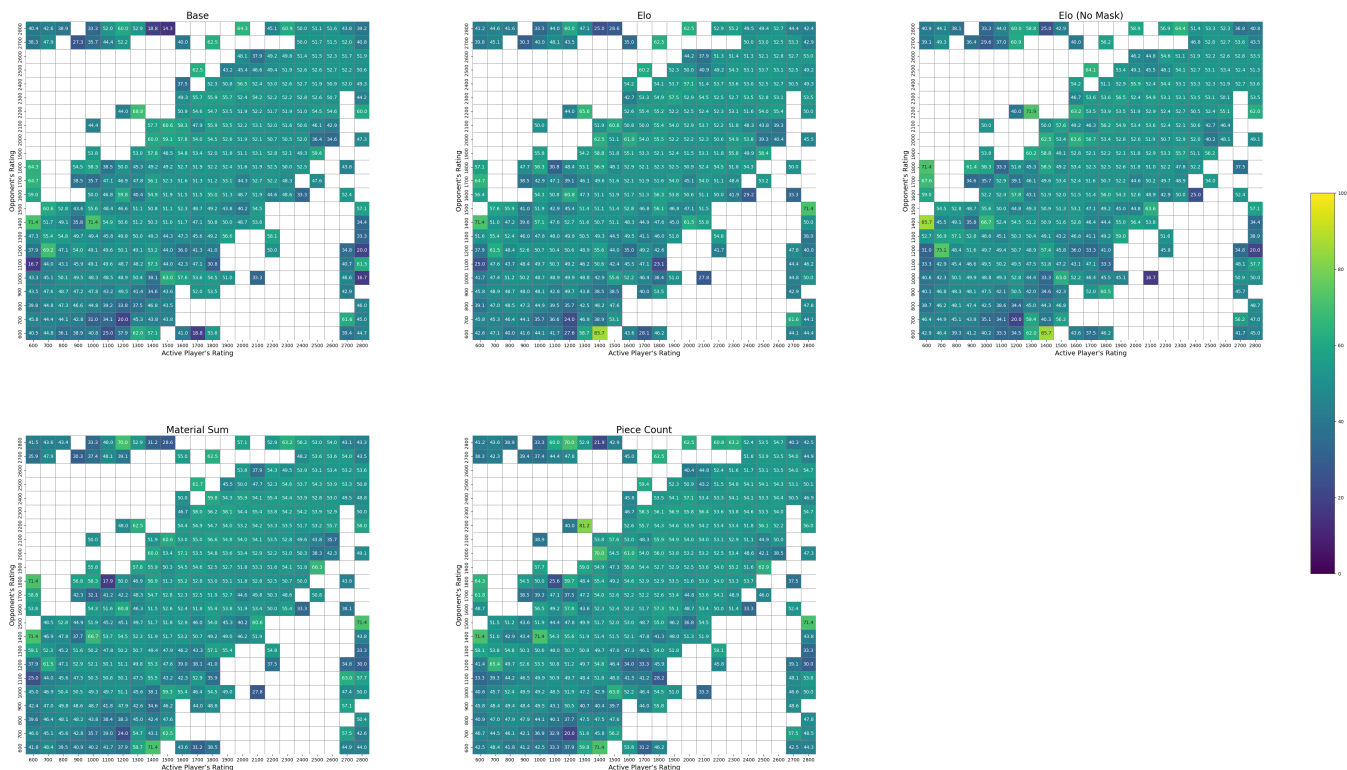


Figure 4: Prediction accuracy across skill levels of predicted player and opponent

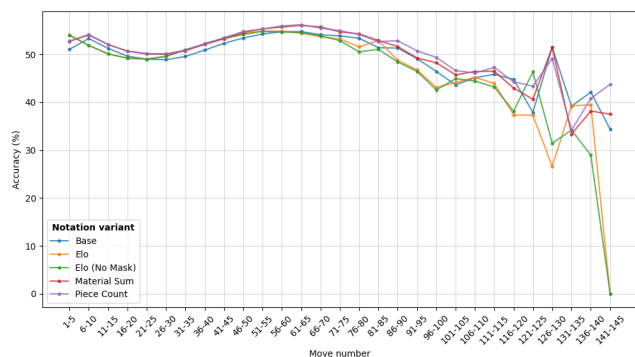


Figure 5: Change in prediction accuracy over time. Each bullet shows the average accuracy across the range of 5 moves (10 ply).

is played by Stockfish in each position, we used Python `stockfish` library with default engine parameters. Then we generated probabilities of each legal move in each position from the games, for each possible rating of the predicted player between 600 and 2800, using 3 models trained with masked Elo. We measured how often the Stockfish move is the one with the highest probability, and what is the average probability of the Stockfish move according to the model. The results are presented in Figures 6 and 7. It can be seen that the higher Elo we set for each model, the more often it plays the same move as Stockfish and the higher probability

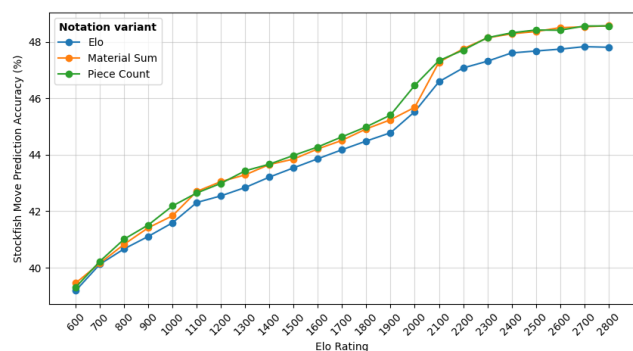


Figure 6: Percentage of predicted moves being the same as Stockfish move across different ratings set to the model.

it assigns to Stockfish's move.

5.3 Combining Language Models with Standard Chess Engines

Although it is known that chess-playing programs are not the best at predicting human moves (McIlroy-Young et al. 2020)), it seems interesting to ask whether a system that aims to play chess in a human-like way could benefit from such programs. This idea is rooted in Daniel Kahneman's theory of System 1 (heuristic, fast) and System 2 (slow, analytical) (Kahneman 2011), according to which human decision-making typically relies on System 1, though the an-

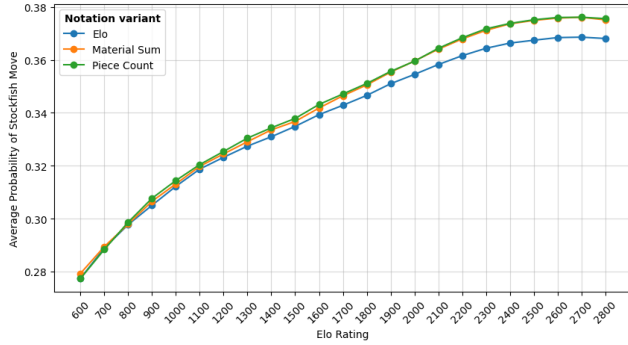


Figure 7: Average probability assigned by the model to a move played by Stockfish, using different values of player rating

alytical system is sometimes also engaged. One may hypothesize that the same holds true for chess players. To test this, we attempt to improve the prediction accuracy of the move played by human players by incorporating analytical chess engines. In this section we present some preliminary experiments based on a selection of games with given Elo between 1500 and 1599.

We use the following agents:

1. A simple MiniMax agent with a Quiescence search, considering only checkmates and material advantage, playing at depth 2 (Agent Naive-2)
2. The same agent, but playing at depth 3 (Agent Naive-3)
3. The Stockfish engine, with the number of calculations drastically limited (to 10,000 nodes)

Each move in a given board state s is characterized by the following variables:

1. The probability given by a language model (LM), $p_{LM}(m|s)$
2. The probability computed using a Softmax function applied to Stockfish evaluation results, $p_{SF}(m|s)$
3. A binary variable: whether move m is the best move according to the Naive-2 agent in state s ($B_2 = 1$ iff $m \in \text{BestMoves}_{N2}(s)$)
4. A binary variable: whether move m is the best move according to the Naive-3 agent in state s ($B_3 = 1$ iff $m \in \text{BestMoves}_{N3}(s)$)

The final value of a move is computed as follows:

$$\text{MoveValue}(s, m) = p_{LM}(m|s) + \alpha_1 p_{SF}(m|s) + \alpha_2 B_2 + \alpha_3 B_3 \quad (1)$$

This formula includes three parameters. We decided to treat this task as a hyperparameter optimization problem, where the objective function is the percentage of correctly predicted moves (parameters are tuned on the training set and evaluated on the test set). The optimization procedure is carried out using the Optuna library (Akiba et al. 2019). The results are presented in Table 5.

Feature used	comment	Dev acc.	Val acc.
N2+N3+SF	no LM	32.3	32.8
LM	baseline	51.95	52.36
LM + N2		52.06	52.49
LM + N3		52.05	52.40
LM + SF		52.16	52.52
LM + N2 + SF		52.47	52.71
LM + N3 + SF		52.47	52.53
LM + N2 + N3 + SF	all data	52.54	52.73

Table 5: Movement prediction accuracy for subcorpus of plays with ELO=1500. Each line contains a different combination of features. Note that there is no overfitting, and validation set seems to be easier. Moreover, both dev set and validation set give roughly the same orders of combinations

It can be seen that both agents that prevent material loss (Naive-2 and Naive-3), as well as the restricted Stockfish engine, play a positive role in modeling player behavior.

6 Conclusion and Future Works

In this work, we have shown that (1) an autoregressive language model trained on sequences of moves can play chess—not only adhering to the rules of the game, but also conducting a “reasonable game” with a level of play that can be easily controlled. Furthermore, (2) inserting single-token information about the game state between moves significantly improves the model’s performance, and (3) the accuracy of predicting human moves can be further increased by combining the language model with minimax-based agents (even very simple ones).

We want to highlight the practical implications of replacing specialized neural networks (which have traditionally been used to create chess-playing agents) with language models. Since our approach does not involve any modifications to the GPT architecture, (1) it can be easily applied to other games, and (2) any improvements to autoregressive language models – such as performance optimizations or replacing the transformer architecture with a newer one – are likely to directly improve the performance of agents based on language models. Additionally, training a transformer network is significantly faster, as the entire game can be given the model at once and processed in parallel.

We also hypothesize that just as we included Elo tokens in the games, it would be possible to introduce “style tokens” (e.g., aggressive, cautious, focused on quick exchanges, or slow positional buildup, etc.). Investigating the modeling of playing style seems to be one of the most interesting directions for future work. Another promising direction is the more effective use of chess agents during training – in particular, replacing the current approach (where the model is first trained and only later combined with other agents) with an approach in which the model learns to leverage information from traditional chess engines during its training.

References

Akiba, T.; Sano, S.; Yanase, T.; Ohta, T.; and Koyama, M. 2019. Optuna: A Next-generation Hyperparameter Opti-

- mization Framework. In *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.
- Bengio, Y.; Ducharme, R.; and Vincent, P. 2000. A Neural Probabilistic Language Model. In Leen, T.; Dietterich, T.; and Tresp, V., eds., *Advances in Neural Information Processing Systems*, volume 13. MIT Press.
- Dauphin, Y. N.; Fan, A.; Auli, M.; and Grangier, D. 2017. Language modeling with gated convolutional networks. In *Proceedings of the 34th International Conference on Machine Learning - Volume 70, ICML'17*, 933–941. JMLR.org.
- David, E.; Netanyahu, N. S.; and Wolf, L. 2017. DeepChess: End-to-End Deep Neural Network for Automatic Learning in Chess. *CoRR*, abs/1711.09667.
- Falcon, W.; and The PyTorch Lightning team. 2019. PyTorch Lightning.
- Hazineh, D. S.; Zhang, Z.; and Chiu, J. 2023. Linear Latent World Models in Simple Transformers: A Case Study on Othello-GPT. arXiv:2310.07582.
- Hsu, F.-H. 1999. IBM's Deep Blue Chess grandmaster chips. *IEEE Micro*, 19(2): 70–81.
- Kahneman, D. 2011. *Thinking, fast and slow*. New York: Farrar, Straus and Giroux. ISBN 9780374275631 0374275637.
- Karpathy, A. 2022. NanoGPT. <https://github.com/karpathy/nanoGPT>.
- Kingma, D. P.; and Ba, J. 2014. Adam: A Method for Stochastic Optimization. *CoRR*, abs/1412.6980.
- Klein, D. 2022. Neural Networks for Chess. arXiv:2209.01506.
- Li, K.; Hopkins, A. K.; Bau, D.; Viégas, F. B.; Pfister, H.; and Wattenberg, M. 2022. Emergent World Representations: Exploring a Sequence Model Trained on a Synthetic Task. *CoRR*, abs/2210.13382.
- McIlroy-Young, R.; Sen, S.; Kleinberg, J.; and Anderson, A. 2020. Aligning Superhuman AI with Human Behavior: Chess as a Model System. In *Proceedings of the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining, KDD '20*, 1677–1687. ACM.
- Nasu, Y. 2018. NNUE: Efficiently Updatable Neural-Network-based Evaluation Functions for Computer Shogi.
- Noever, D.; Ciolino, M.; and Kalin, J. 2020. The Chess Transformer: Mastering Play using Generative Language Models. arXiv:2008.04057.
- Pascutto, Gian-Carlo and Linscott, Gary. 2019. Leela Chess Zero.
- Radford, A.; and Narasimhan, K. 2018. Improving Language Understanding by Generative Pre-Training.
- Ramesh, A.; Pavlov, M.; Goh, G.; Gray, S.; Voss, C.; Radford, A.; Chen, M.; and Sutskever, I. 2021. Zero-Shot Text-to-Image Generation. *CoRR*, abs/2102.12092.
- Silver, D.; Hubert, T.; Schrittwieser, J.; Antonoglou, I.; Lai, M.; Guez, A.; Lanctot, M.; Sifre, L.; Kumaran, D.; Graepel, T.; Lillicrap, T.; Simonyan, K.; and Hassabis, D. 2018. A general reinforcement learning algorithm that masters chess, shogi, and Go through self-play. *Science*, 362(6419): 1140–1144.
- Tang, Z.; Jiao, D.; McIlroy-Young, R.; Kleinberg, J.; Sen, S.; and Anderson, A. 2024. Maia-2: A Unified Model for Human-AI Alignment in Chess. arXiv:2409.20553.
- Toshniwal, S.; Wiseman, S.; Livescu, K.; and Gimpel, K. 2022. Chess as a Testbed for Language Model State Tracking. arXiv:2102.13249.
- Vaswani, A.; Shazeer, N. M.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, L.; and Polosukhin, I. 2017. Attention is All you Need. In *Neural Information Processing Systems*.
- Wu, H.; Xu, J.; Wang, J.; and Long, M. 2021. Autoformer: decomposition transformers with auto-correlation for long-term series forecasting. In *Proceedings of the 35th International Conference on Neural Information Processing Systems, NIPS '21*. Red Hook, NY, USA: Curran Associates Inc. ISBN 9781713845393.
- Wu, S.; Li, X.; Yu, F.; and Sun, M. 2023. TunesFormer: Forming Irish Tunes with Control Codes by Bar Patching. In Porcaro, L.; Battle-Roca, R.; and Gómez, E., eds., *Proceedings of the 2nd Workshop on Human-Centric Music Information Retrieval 2023 co-located with the 24th International Society for Music Information Retrieval Conference (ISMIR 2023), Milan, Italy, November 10, 2023*, volume 3528 of *CEUR Workshop Proceedings*. CEUR-WS.org.
- Zaremba, W.; Sutskever, I.; and Vinyals, O. 2014. Recurrent Neural Network Regularization.